VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY

Ho Chi Minh City University of Technology

Faculty of Computer Science and Engineering



Deep Learning and Its Applications (CO3133)

Assignment 1 Report

Foundations of Deep Learning Pipelines and Architectures

From Linear Models to Modern Sequence Models:
A Comparative Study for Image Classification

Instructor: Le Thanh Sach

Semester: 261
Group: 18 – DeepDive
Students: Le Pham Tien Long – 2352688
Ngo Tieu Nghi – 2352799
Ho Minh Nhat – 2352858

HO CHI MINH CITY, 2026

Contents

List of Figures **2**

List of Tables **2**

1 Problem and Data Description **3**

1.1 Problem formulation . . . 3
1.2 Dataset description . . . 3
1.3 Exploratory data analysis . . . 4
1.4 Data splitting . . . 5
1.5 Preprocessing . . . 6

2 Methodology **6**

2.1 Pipeline overview . . . 6
2.2 Baseline and main methods . . . 7
2.3 Training setup . . . 8
2.4 Experimental design . . . 8

3 Implementation Results **8**

3.1 Training behaviour . . . 8
3.2 Quantitative results . . . 9
3.3 Qualitative results . . . 10
3.4 Comparison and discussion . . . 11
3.5 Error analysis . . . 11
3.6 Ablation study . . . 12
3.6.1 Sequence representation . . . 12
3.6.2 Recurrent cell . . . 13
3.6.3 Effect on the main comparison . . . 13
3.7 Limitations and conclusion . . . 14

4 AI Usage Disclosure **14**

Code Availability **16**

References **16**

List of Figures

1 Images per class in each subset. . . . 3
2 Eight training examples per class. . . . 4
3 Class mean similarity, left, and the nearest-class-mean confusion matrix, right. . . . 5
4 Signal per timestep, and per 4×4 patch token. . . . 5
5 Pipeline, from the raw dataset to an evaluated checkpoint. . . . 6
6 Learning curves per architecture, averaged over three seeds. . . . 9
7 Confusion matrix of the CNN on the test split, rows normalised to percent. . . . 10
8 CNN predictions on the test split: correct above, errors below. . . . 10
9 Learning curves for the six representation configurations. . . . 13

List of Tables

1 Split sizes. . . . 5
2 Shared training protocol. . . . 8
3 Architecture comparison. . . . 9
4 Error groups for the CNN on the test split. . . . 11
5 Sequence representation. . . . 12
6 Recurrent cell at the row representation. . . . 13

1 Problem and Data Description

1.1 Problem formulation

Automated garment recognition underpins catalogue tagging, visual search and stock auditing in online retail, where a product photograph must be routed to a category without a human in the loop. Fashion-MNIST [1] is the standard small benchmark for this setting, cheap enough to train many architectures under one protocol.

The task is single-label image classification, and the prediction unit is one image. The input is a grayscale image $x \in \mathbb{R}^{1\times 28\times 28}$ and the output is one of $C = 10$ garment classes. Every model implements a function f_θ returning raw logits $z = f_\theta(x) \in \mathbb{R}^C$, from which the prediction is $\hat{y} = \arg\max_i z_i$. Training minimises the cross-entropy between the logits and the true label y over the training set $\mathcal{D}$:

$$\mathcal{L}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{(x,y)\in\mathcal{D}} \log \frac{\exp(z_y)}{\sum_{i=1}^{C} \exp(z_i)}. \tag{1}$$

The softmax of Equation 1 is folded into the loss, so no model applies an explicit softmax before it.

Two difficulties dominate. The four upper-body garments share a silhouette that survives poorly at 28×28 grayscale, which Section 1.3 shows is measurable before any training. And because the comparison itself is the object of study, every difference in score must trace to the architecture rather than to the data, the split or the tuning budget, which constrains the design far more than the classification task does.

1.2 Dataset description

Fashion-MNIST was released by Zalando Research [1] as a drop-in replacement for MNIST, under the MIT licence. It is obtained through `torchvision.datasets.FashionMNIST`, which downloads the four original IDX binary files, roughly 30 MB uncompressed. The images are centred 28×28 grayscale thumbnails from Zalando's product catalogue, labelled by the catalogue category of the article, so the labels come from the retailer rather than from crowd workers.

The corpus holds 70,000 images, 60,000 in the official training set and 10,000 in the official test set, with exactly 6,000 and 1,000 images per class respectively. The class distribution is therefore uniform by construction, as Figure 1 confirms, and the max-to-min per-class ratio is 1.000. No imbalance correction is required, and accuracy and macro-F1 are not distorted by class priors.

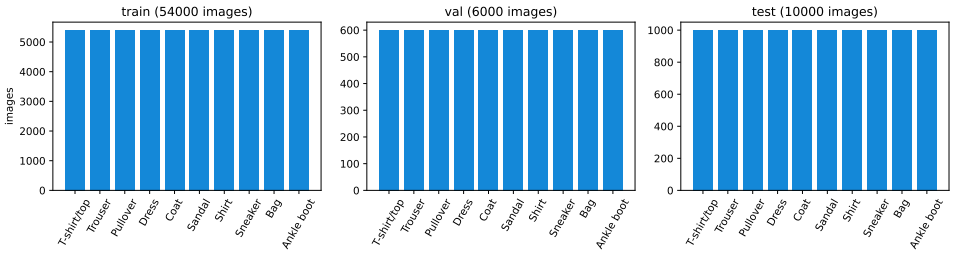


Figure 1: *Images per class in each subset.*

Three limitations bound what these results can claim. The images are grayscale, square and centred, so the models never face colour, clutter, occlusion or scale variation. The resolution discards the fabric texture and stitching that distinguish a shirt from a coat in a real catalogue. And the labels follow one retailer's merchandising taxonomy, in which the boundary between a pullover, a coat and a shirt is a commercial convention rather than a visual fact, so part of the residual error is irreducible.

1.3 Exploratory data analysis

Every image has the same input size, one channel of 28×28 `uint8` pixels, so no resizing or padding is needed. Half of all training pixels are exactly zero and the garment is centred, so much of the 784 flattened features carries no signal. Figure 2 shows the variation within each class: footwear and bags are distinctive, while the upper-body garments share an outline.

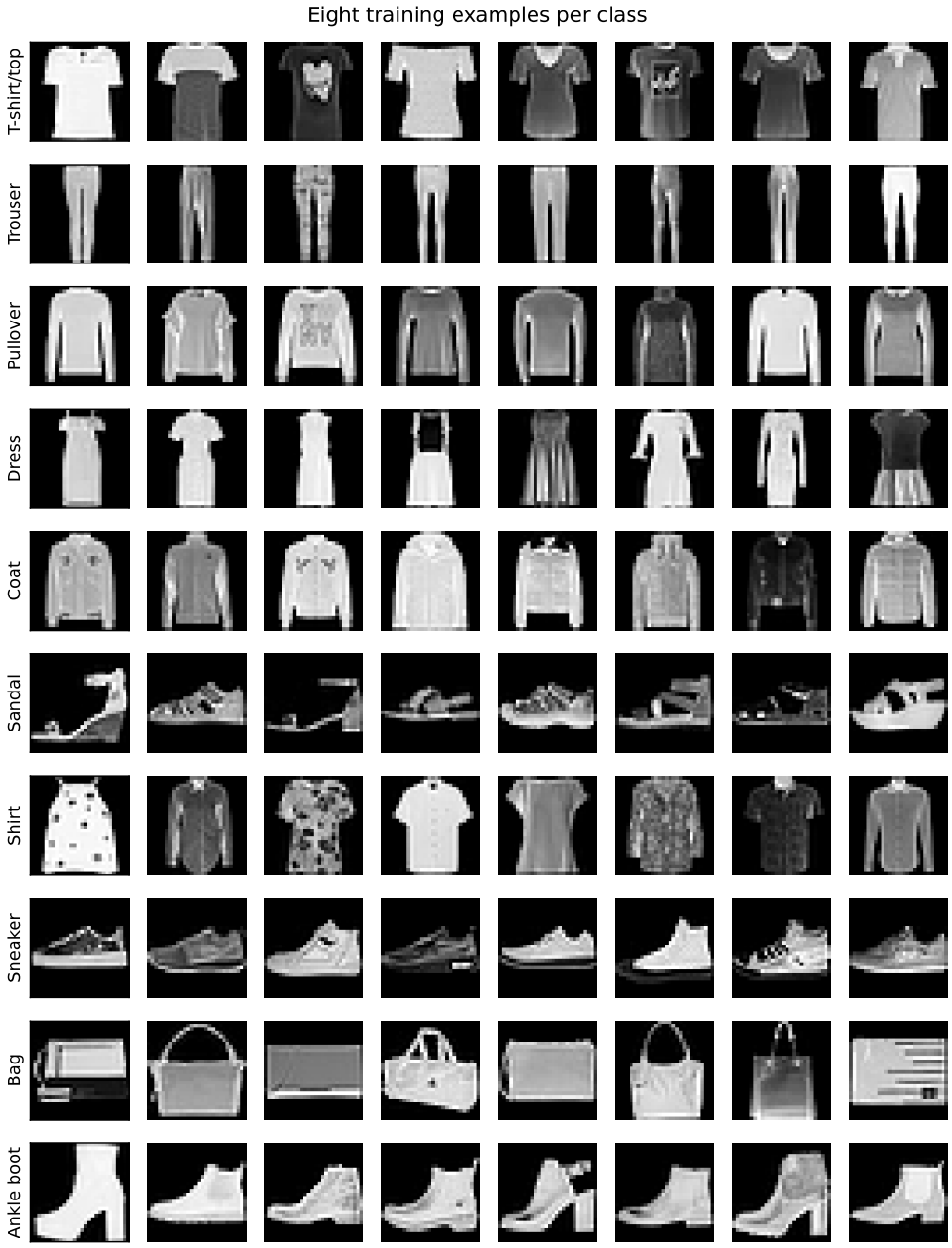


Figure 2: *Eight training examples per class.*

Class overlap is measurable before any model is trained. Cosine similarity between class mean images exceeds 0.98 for Coat/Shirt, Pullover/Coat and Pullover/Shirt, and a nearest-class-mean classifier on raw pixels reaches 68.4% validation accuracy with per-class recall from 0.220 for Shirt to 0.872 for Ankle boot. Figure 3 reports both, and they isolate the same cluster: a floor the trained models must clear, and a prediction of where they will fail that Section 3.5 confirms.

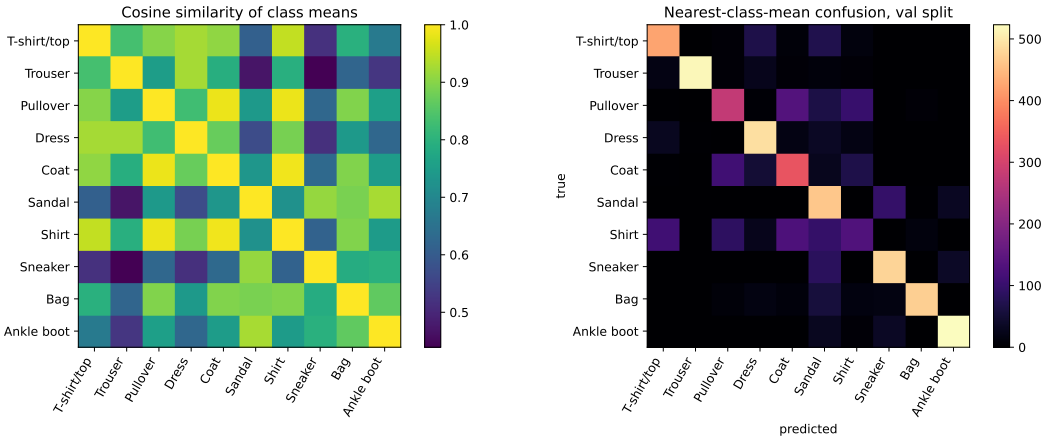


Figure 3: *Class mean similarity, left, and the nearest-class-mean confusion matrix, right.*

The flattened view is highly redundant: ten principal components explain 72.1% of pixel variance and one hundred explain 91.4%. Pixel correlation falls from 0.747 at a distance of one pixel to 0.159 at ten and 0.093 at twenty. That local structure is what a convolution exploits and a flattened view discards, a concrete reason to expect the architectures to separate.

The same image also supports three sequence views, used by the recurrent and Transformer models: rows and columns give 28 timesteps of 28 features, 4×4 patches give 49 tokens of 16. Figure 4 shows how unevenly each spreads its signal. No row falls below a quarter of peak signal, against one column and three patch tokens, so patch tokenisation concentrates background into a few weak tokens while rows and columns spread it across every step.

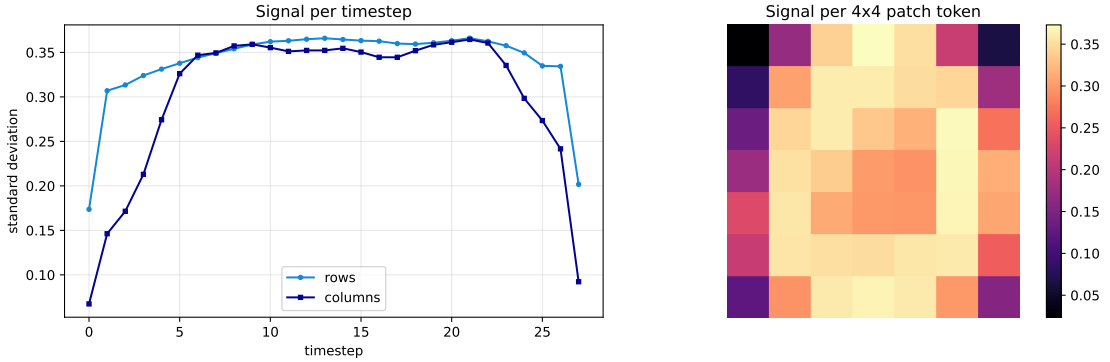


Figure 4: *Signal per timestep, and per 4×4 patch token.*

1.4 Data splitting

The official test set of 10,000 images is held out untouched. The official 60,000 training images are split once with `train_test_split` at seed 0, stratified by class, giving the sizes in Table 1. The split unit is the image, which is also the prediction unit, and the images are independent product photographs, so no group structure can leak across the boundary.

Table 1: *Split sizes.*

Subset	Images	Per class
Train	54,000	5,400
Validation	6,000	600
Test	10,000	1,000

The indices are generated once and committed, and every model loads the same file, so the split is

identical across architectures, seeds and machines. Two rules prevent leakage. Normalization statistics are computed on the training split alone and recorded with each run, never recomputed from the data a model is evaluated on. The test split is read only after model selection has concluded, and once per configuration, so it never participates in any decision.

1.5 Preprocessing

The corpus needs no cleaning: every image is complete, every label is a valid class index, and there are no invalid annotations to handle. No resizing or padding is applied, since the input size is already uniform.

Images are converted to `float32` in $[0, 1]$ and standardised with the training-split mean $\mu = 0.2858$ and standard deviation $\sigma = 0.3529$, so the network input is $(x/255 - \mu)/\sigma$. Augmentation applies to the training loader only: a random 28×28 crop after 2 pixels of edge padding, then a horizontal flip with probability 0.5. Validation and test loaders stay deterministic. The sequence models receive the same normalised tensor and reshape it internally, to 28×28 for rows and 49×16 for patch tokens, so tokenisation never changes the pixels.

A batch leaving the training loader is a pair of tensors: images of shape $(128, 1, 28, 28)$ in `float32` spanning $[-0.810, 2.024]$, and labels of shape (128) in `int64` spanning $[0, 9]$. The observed range is consistent with standardisation, since $(0 - \mu)/\sigma = -0.810$ is the value of a black pixel and $(1 - \mu)/\sigma = 2.024$ that of a white one.

2 Methodology

2.1 Pipeline overview

Figure 5 shows the path from the raw dataset to an evaluated checkpoint. All five models take the same path; only the model block changes, which is what makes the comparison in Section 3 attributable to the architecture.

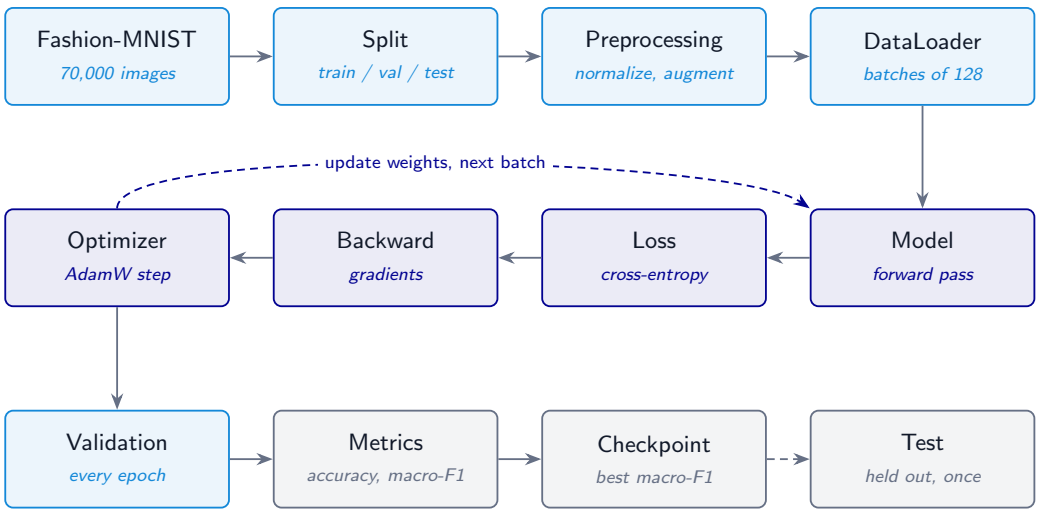


Figure 5: *Pipeline, from the raw dataset to an evaluated checkpoint.*

2.2 Baseline and main methods

All five share the loss of Equation 1 and are scored with accuracy and macro-F1. They differ only in how they represent the input and what structure they assume.

Linear classifier (baseline). The image is flattened to $x \in \mathbb{R}^{784}$ and mapped to logits by one affine transformation,

$$z = Wx + b, \qquad W \in \mathbb{R}^{10\times 784},\ b \in \mathbb{R}^{10}, \tag{2}$$

giving 7,850 parameters. The raw logits go straight into the cross-entropy loss, with no softmax in front of it. It can draw only one hyperplane per class.

Multilayer perceptron (MLP). Two hidden layers add the non-linear pixel interaction that Equation 2 cannot express:

$$\begin{aligned} h_1 &= \mathrm{Dropout}_{0.2}(\mathrm{ReLU}(W_1 x + b_1)),\\ h_2 &= \mathrm{Dropout}_{0.2}(\mathrm{ReLU}(W_2 h_1 + b_2)),\\ z &= W_3 h_2 + b_3, \end{aligned} \tag{3}$$

with widths $784 \to 512 \to 256 \to 10$ and 535,818 parameters. The activation $\mathrm{ReLU}(a) = \max(0, a)$ does not saturate, so gradients reach both hidden layers without the vanishing behaviour of a sigmoid. Dropout at $p = 0.2$ after each hidden layer, active in training only, forces each unit to stay useful when a random fifth of its neighbours is removed. Its input is a flat vector, so it has no access to spatial structure.

Convolutional network (CNN). Designed for this task rather than taken from a reference model. A convolution applies a small filter at every position, so the same local pattern is recognised wherever it occurs and the parameter count stops scaling with image size:

$$\begin{aligned} f_1 &= \mathrm{MaxPool}_2(\mathrm{ReLU}(W_1 * x + b_1)),\\ f_2 &= \mathrm{MaxPool}_2(\mathrm{ReLU}(W_2 * f_1 + b_2)),\\ z &= W_4\,\mathrm{ReLU}(W_3\,\mathrm{vec}(f_2) + b_3) + b_4. \end{aligned} \tag{4}$$

Each filter is 3×3 with padding 1, so convolution preserves spatial extent and only pooling reduces it, $28 \times 28 \to 14 \times 14 \to 7 \times 7$. Each output channel is a feature map recording where that filter responded; channels rise $1 \to 32 \to 64$ as resolution falls, trading spatial precision for more patterns. Max pooling keeps the strongest response in each 2×2 window, halving the resolution and tolerating small translations. The flattened $64 \times 7 \times 7$ map feeds a 128-unit hidden layer with dropout 0.2, for 421,642 parameters.

Recurrent network (GRU). The image is read as a sequence of its 28 rows: one timestep per row, each an input vector of size 28. A single-layer GRU [2] with hidden size 128 carries a state h_t updated at every row, so h_t summarises the image from the top edge down to row t and the final state h_{28} represents the whole image:

$$h_t = \mathrm{GRU}(x_t, h_{t-1}), \qquad z = W\,\mathrm{LayerNorm}(h_{28}) + b. \tag{5}$$

At 62,218 parameters it is the smallest model with any depth, and it reaches vertical structure only through the order of its timesteps.

Transformer. The image is split into 49 non-overlapping 4×4 patches, each flattened to 16 values and projected by a learned matrix E to a 128-dimensional token. A learned class token is prepended and a learned positional embedding P added, which is necessary because self-attention is permutation-invariant and would otherwise discard the arrangement of the patches:

$$H_0 = [\,x_{\mathrm{cls}};\, Ep_1;\, \ldots;\, Ep_{49}\,] + P, \qquad z = W\,H_L^{(\mathrm{cls})} + b. \tag{6}$$

Four pre-norm encoder layers [3] follow, each with four-head self-attention over all 50 tokens and a 256-unit feed-forward block with dropout 0.1. Attention takes the token sequence and returns one of the same length, each output token a weighted mixture of all inputs, so a patch can attend to any other regardless of distance. The class token gives the logits, for 540,170 parameters. It assumes no locality at all.

Implementation. The five architectures, the registry that selects them, the training loop, the evaluation and the figures are written by the group. Layer primitives (`nn.Linear`, `nn.Conv2d`, `nn.MaxPool2d`, `nn.GRU`, `nn.TransformerEncoder`, `nn.Dropout`, `nn.LayerNorm`), the optimizers and the scheduler come

from PyTorch [4], the dataset loader from torchvision, the metrics and the stratified split from scikit-learn, and the plots from matplotlib. No pretrained weights are used; every model starts from random initialisation.

2.3 Training setup

Every model is trained under the identical protocol of Table 2. Hyperparameters are not tuned per architecture: one configuration was fixed in advance from values conventional for AdamW [5] on a small image benchmark, then applied unchanged to all five, since tuning each separately would confound the comparison with unequal search effort. Each score is therefore the score under a common budget, not the best that architecture could reach.

Table 2: *Shared training protocol.*

Setting	Value
Optimizer	AdamW, $\eta = 3 \times 10^{-4}$, weight decay $\lambda = 0.05$
Batch size	128
Epochs	30 maximum
Scheduler	Cosine annealing over 30 epochs
Regularization	Weight decay; dropout where the architecture defines it; random crop and horizontal flip on the training loader only
Early stopping	After 5 epochs without an improvement in validation macro-F1
Checkpoint criterion	Highest validation macro-F1
Seeds	0, 1, 2 for every configuration
Hardware	Apple M4 Pro, 14 cores, 48 GB unified memory, MPS backend
Software	Python 3.12.0, PyTorch 2.14.0, torchvision 0.29.0, scikit-learn 1.9.1
Mixed precision	Not used; all runs are full `float32`

Each seed sets the Python, NumPy and PyTorch generators together, and every run directory records the resolved configuration, the environment and the git commit, so any reported number traces back to the run that produced it. The fifteen runs took 1.76 hours, from 258 s for the Linear baseline to 690 s for the Transformer.

2.4 Experimental design

The main experiment varies the architecture over its five levels and fixes everything else: the committed split, the protocol of Table 2 and the seed set. Its hypothesis is that inductive bias, not parameter count, governs macro-F1 here, and macro-F1 decides it. Because each sequence model is compared at its own default view of the image, the design confounds the architecture with that view; the ablations of Section 3.6 separate the two under the same criterion.

The conclusion criterion was fixed before the results were seen and applies to every comparison in this report. Configuration A is better than B only when the mean validation macro-F1 of A exceeds that of B and the two intervals of mean $\pm$ one standard deviation across the three seeds do not overlap. Overlapping intervals mean the two are indistinguishable, which is itself a result and is not resolved by picking the higher mean.

3 Implementation Results

3.1 Training behaviour

Figure 6 shows the learning curves, each averaged over three seeds. The ordering visible at epoch 30 is already established by epoch 10 and does not cross afterwards.

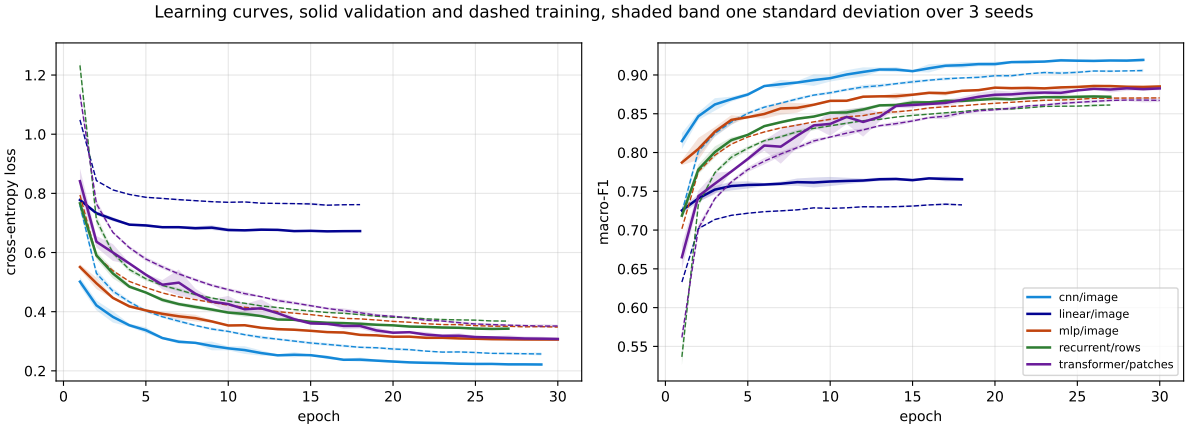


Figure 6: *Learning curves per architecture, averaged over three seeds.*

Validation scores better than training, not worse: for the CNN, MLP and Linear baseline, validation loss is below training loss and validation macro-F1 above it at every epoch of every seed, and the Transformer and GRU agree until individual seeds cross over in the final third. This is not leakage. Augmentation and dropout are active in the training pass only, so training metrics are measured on a deliberately harder task; the gap is the cost of that regularization.

None of the five overfits: validation loss is still at its minimum in the final epoch for every architecture, exceeding it by at most 0.0008. The Linear baseline is the one clear case of underfitting, plateauing near 0.77 macro-F1 while its training loss stays high. Early stopping fired only for that baseline, at 24.3 epochs on average; the other four ran the full 30 with the best epoch between 25.3 and 27.3, so the budget was adequate but not generous.

Three implementation faults were found and fixed, each of which would have corrupted the comparison. The run directory was written before the normalization statistics were computed, so the recorded configuration did not describe the run. The loader then recomputed those statistics instead of reusing the recorded ones, found by tampering with the recorded value and seeing it silently overwritten. Per-run metrics were rounded before averaging across seeds, inflating the measured standard deviation by up to 26.8%.

3.2 Quantitative results

Table 3 reports every mandatory comparison quantity as a mean over seeds 0, 1 and 2 with one standard deviation. Inference time is per image at batch size 1, and the fifteen runs were trained consecutively on the one idle machine of Table 2, so the times compare with each other but do not transfer. Accuracy agrees with macro-F1 to within 0.0013 for every model, as expected on a balanced split.

Table 3: *Architecture comparison.*

Model	Parameters	s/epoch	ms/image	Val macro-F1	Test macro-F1	Test accuracy
CNN	421,642	13.2	0.305	**0.9198 ± 0.0018**	**0.9110 ± 0.0007**	**0.9112 ± 0.0006**
MLP	535,818	11.3	0.128	0.8862 ± 0.0015	0.8807 ± 0.0018	0.8810 ± 0.0020
Transformer	540,170	23.0	1.090	0.8834 ± 0.0023	0.8796 ± 0.0011	0.8804 ± 0.0014
GRU	62,218	14.7	2.069	0.8731 ± 0.0012	0.8667 ± 0.0013	0.8669 ± 0.0015
Linear	7,850	10.6	0.036	0.7697 ± 0.0025	0.7619 ± 0.0021	0.7631 ± 0.0025

The spread across seeds is at most 0.0025 macro-F1, so the ranking is stable under reinitialisation. Validation overstates test performance for every architecture, by at most 0.0088 macro-F1, but the ordering is unchanged: selecting on validation did not select a model that fails to generalise. The gap is the price of having consulted validation four times over, for monitoring, checkpointing, early stopping and the choice of architecture.

3.3 Qualitative results

Figure 7 gives the confusion matrix of the CNN, rows normalised because the errors are distributed very unevenly. The diagonal dominates, and the only substantial off-diagonal mass forms a single block among T-shirt/top, Pullover, Coat and Shirt.

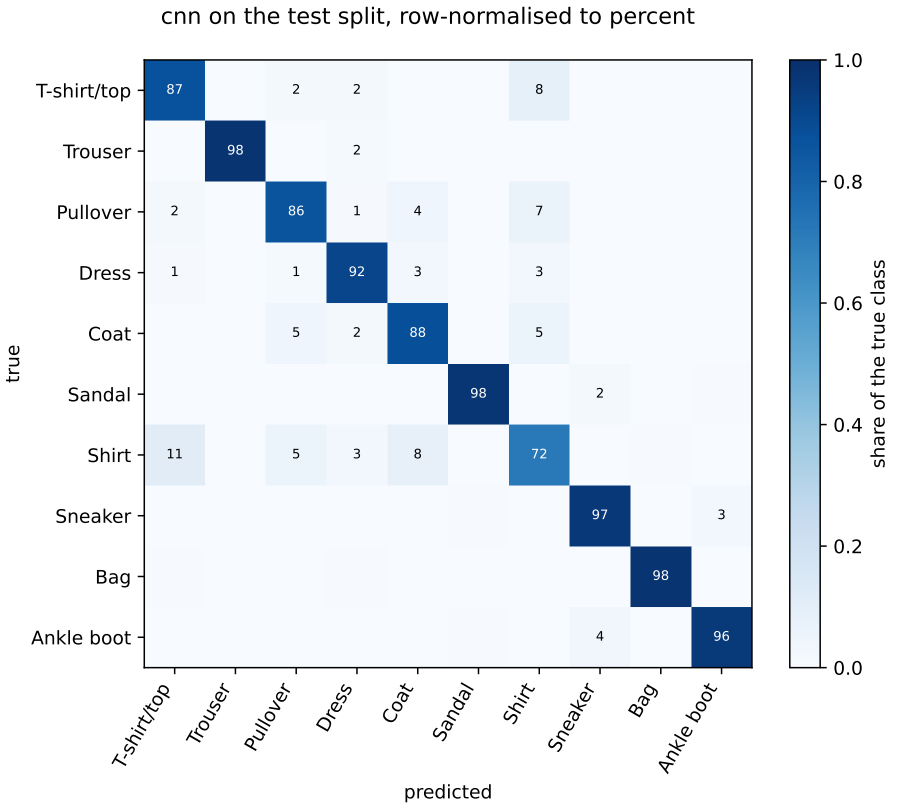


Figure 7: *Confusion matrix of the CNN on the test split, rows normalised to percent.*

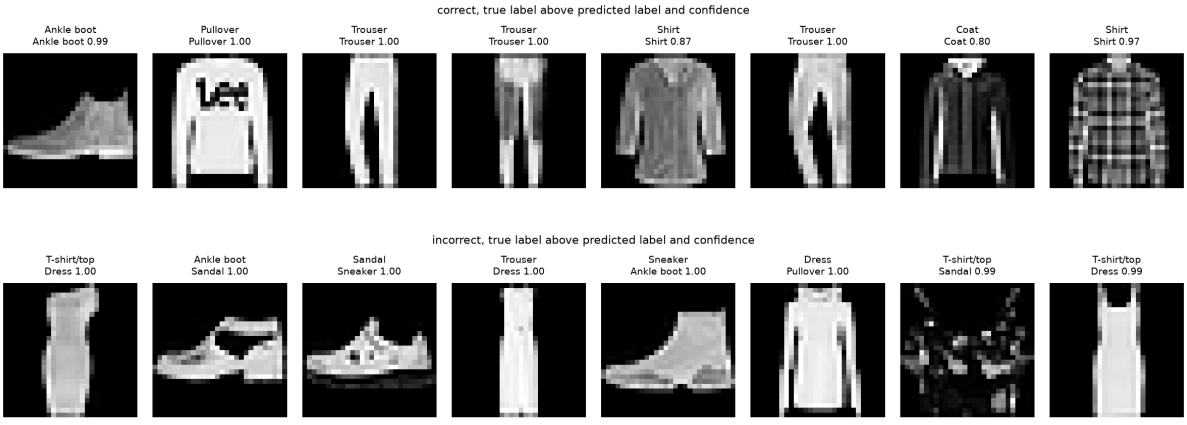


Figure 8: *CNN predictions on the test split: correct above, errors below.*

Figure 8 places predictions against ground truth: the upper row is the first eight test images in loader order, the lower the eight errors the model was most confident about. The upper row holds two genuinely difficult cases the model still resolves, a plaid Shirt at confidence 0.97 and a dark hooded Coat at 0.80, both recognised from texture and silhouette rather than brightness. The lower row is more informative, because every image in it fails at confidence 0.99 or above: a sleeveless garment labelled T-shirt/top but

predicted Dress, a flat open-backed shoe labelled Ankle boot but predicted Sandal, a Trouser predicted Dress. The model is not uncertain about these; the image is simply compatible with both labels. They mark a ceiling no architecture compared here can pass.

3.4 Comparison and discussion

Applying the criterion of Section 2.4 to Table 3, the CNN beats all four alternatives, the MLP and the Transformer are indistinguishable from each other, both beat the GRU, and all four beat the Linear baseline. The CNN's margin on test is 0.0303 macro-F1 over the MLP and 0.1491 over the baseline. The comparison is admissible because every model saw the identical committed split under the protocol of Table 2, with the same seeds, checkpoint rule, hardware and library versions.

The ranking does not follow capacity. The CNN wins with 21% fewer parameters than the MLP and 22% fewer than the Transformer, and the Transformer, the largest and most expensive to train, does not separate from an MLP that trains in half the time. What distinguishes the five is the inductive bias each imposes on the same pixels, which follows from the input representation. The CNN keeps the image as a grid and assumes locality and translation equivariance, matching the pixel correlation of Section 1.3; the weight sharing this implies is why it is at once the most accurate and the smallest of the three strong models. The MLP receives a flat vector in which neighbouring pixels are no longer neighbours, so it learns that structure from data and pays in parameters. The GRU sees rows in sequence, capturing real vertical structure but asserting a processing order a static image does not have. The Transformer has no locality prior, so attention must discover neighbourhood relations from data, and at this scale it does not close the gap. The baseline's 0.7619 measures directly that the classes are not linearly separable in pixel space.

Accuracy is not the only axis. The GRU reaches 0.8667 with under an eighth of the MLP's parameters, trailing it by 0.0140, yet is slowest at inference, 2.069 ms per image against the CNN's 0.305, because its timesteps are sequential while convolution is not; parameters and latency are not interchangeable proxies for cost. Among the models competitive on accuracy the CNN is also the cheapest to run, so here there is no accuracy-against-speed trade-off.

The hypothesis that inductive bias rather than capacity governs performance is therefore supported, but two conclusions do not follow. The first is that attention is the reason the Transformer trails: the ablation of Section 3.6 shows that most of its deficit against the MLP comes from the patch tokenisation it was given, not from the architecture, and that the same model on row tokens beats the MLP outright. The attribution above therefore holds for the Transformer only as configured here. The second is that any single component of the CNN causes its advantage; separating locality from weight sharing from pooling would need a further ablation that was not run.

3.5 Error analysis

The CNN misclassifies 888 of the 10,000 test images, an error rate of 8.9% averaged over three seeds. Table 4 splits them into three groups of very unequal size.

Table 4: *Error groups for the CNN on the test split.*

Group	Errors	Share	Description
Upper-body garments	576	64.9%	Confusions among T-shirt/top, Pullover, Coat and Shirt
Footwear	93	10.5%	Confusions among Sandal, Sneaker and Ankle boot
Scattered	219	24.6%	Remainder, of which 81% involve Dress
Total	888	100%	

Upper-body garments. Almost two thirds of all errors, every confusion bidirectional. The largest flows are Shirt predicted as T-shirt/top, 118 of 1,000 Shirts, then T-shirt/top as Shirt, 78, and Shirt as Coat, 75. Shirt is the only class below 0.80 F1, at 0.7365 with precision 0.753 and recall 0.721, and it is the lowest class for all five architectures. The cause lies in the data: these classes are separated by sleeve length, collar shape and fabric texture, and Figure 3 showed before any training that they have

the most similar class means. The same block holds 64% of the MLP's errors, 64% of the GRU's, 61% of the Transformer's and 48% of the Linear baseline's, so it is a property of the task at this resolution rather than of any one architecture. The baseline is the exception that confirms it: its errors are the least concentrated only because it also makes 2,369 of them, spreading failure over classes the other four separate easily.

Footwear. Only 93 errors across three close classes. Sole shape and ankle height survive downsampling far better than fabric texture, which is why all three exceed 0.95 F1 and why this group is not worth attacking.

Scattered errors. The remaining quarter is spread thinly and dominated by Dress, involved in 81% of it, chiefly against Shirt and Coat. It includes the confident failures of Figure 8, where the annotation is arguably as much at fault as the prediction.

Only the first group is worth improving, and capacity is not the way: the Transformer carries 28% more parameters than the CNN and a worse Shirt F1. A class-weighted or focal loss would trade precision on the three easier classes for recall on Shirt, where macro-F1 is lost. Augmentation targeted at sleeve and collar geometry would make the separating cues more salient than a random crop does. A second-stage classifier on the four-class block would specialise on the 65% of errors that matter.

3.6 Ablation study

The comparison above gives each sequence model its own default view of the image, so it confounds the architecture with that view. Two ablations separate them. Both hold the protocol of Table 2 and the three seeds fixed, both are judged on validation by the criterion of Section 2.4, and the test split is not touched.

3.6.1 Sequence representation

The first ablation fixes the architecture and varies only how the image is turned into a sequence: 28 rows of 28 features, 28 columns of 28, or 49 patches of 16. Table 5 and Figure 9 report the result.

Table 5: *Sequence representation.*

Model	Representation	Tokens	Parameters	s/epoch	Val macro-F1
GRU	rows	28×28	62,218	14.7	$\mathbf{0.8731 \pm 0.0012}$
GRU	columns	28×28	62,218	14.6	0.8726 ± 0.0017
GRU	4×4 patches	49×16	57,610	17.8	0.8215 ± 0.0007
Transformer	rows	28×28	539,018	18.4	$\mathbf{0.8987 \pm 0.0024}$
Transformer	columns	28×28	539,018	18.4	0.8961 ± 0.0014
Transformer	4×4 patches	49×16	540,170	23.0	0.8834 ± 0.0023

The hypothesis that patch tokens would win, because a patch preserves two-dimensional locality, is refuted for both architectures: patches are the worst of the three every time. Rows and columns are indistinguishable from each other, which is expected, since these garments are roughly symmetric about the vertical axis and neither scan direction sees more structure than the other.

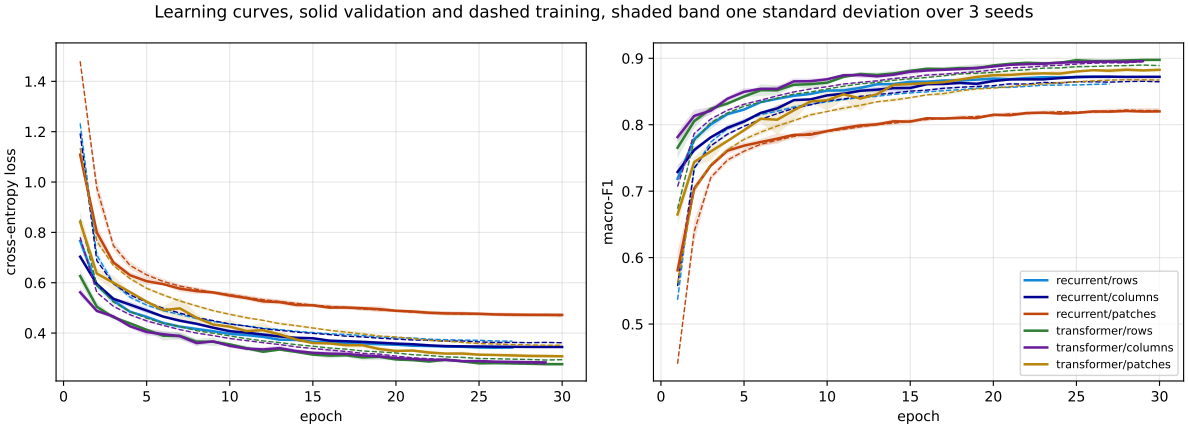


Figure 9: *Learning curves for the six representation configurations.*

Figure 4 explains why. Three patch tokens fall below a quarter of peak signal against no row, because patch tokenisation gathers the empty border into a few near-constant tokens; a patch sequence carries the same information in more tokens, each narrower and several nearly empty. The cost is very unequal, -0.0516 macro-F1 for the GRU against -0.0153 for the Transformer, and the asymmetry follows from how each reaches distant information. Attention addresses any token directly, so lengthening the sequence from 28 to 49 costs it little, whereas recurrence carries state across every added step, and 75% more steps of a weaker signal is where the GRU loses. Patches are also slower for both, because sequence length sets the number of sequential steps.

3.6.2 Recurrent cell

The second ablation fixes the row representation and varies only the gating, between the GRU used throughout and an LSTM of the same hidden size and depth.

Table 6: *Recurrent cell at the row representation.*

Cell	Parameters	s/epoch	ms/image	Val macro-F1	Val accuracy
LSTM	82,442	12.7	0.630	$\mathbf{0.8767 \pm 0.0011}$	$\mathbf{0.8773 \pm 0.0013}$
GRU	62,218	14.7	2.180	0.8731 ± 0.0012	0.8734 ± 0.0014

The hypothesis that the GRU would match the LSTM at lower capacity is refuted on both counts, as Table 6 shows. The LSTM is better, by 0.0036 macro-F1 with non-overlapping intervals, and cheaper to run despite 33% more parameters. The margin is small and bought with those parameters, so it says only that the separate cell state helps slightly over 28 steps. The timing should be read with care: an LSTM does strictly more arithmetic per step, so the 14% faster epoch and 3.5× faster inference are properties of how well each fused kernel is optimised on this backend, not of the algorithms, and nothing here measures the kernels directly.

3.6.3 Effect on the main comparison

The representation ablation revises one conclusion of Section 3.4. On row tokens the Transformer reaches 0.8987 ± 0.0024, whose interval clears the MLP's 0.8862 ± 0.0015, so the two are no longer indistinguishable: the Transformer beats the MLP once it is given a representation that suits it. The tie reported in Table 3 is thus an artefact of the patch tokenisation, not evidence that attention cannot exceed a dense network at this scale.

What survives unchanged is the headline. The CNN at 0.9198 ± 0.0018 still beats the best sequence configuration found here by a clear margin, with fewer parameters than either sequence model, so inductive bias rather than capacity remains the better explanation of the ranking. The GRU's choice of rows, made before these experiments, also turns out to have been its best option.

3.7 Limitations and conclusion

Four limits bound these results. The comparison uses one dataset of 28×28 grayscale images with balanced classes, so neither the ranking nor the error structure should be assumed to carry over to natural images, higher resolutions or imbalanced data. Three seeds estimate a standard deviation from three observations, so a tie means two configurations were not separated at this sample size, not that they are equal. All five share one optimizer, schedule and budget, as fairness demands, so each figure is a score under a common protocol rather than the best each architecture could reach. No pretrained weights are used, which constrains the Transformer most.

The research question was whether architectural inductive bias or capacity governs performance here. The evidence favours inductive bias: the most accurate model is the smallest of the three strong ones, it keeps its margin even over the best sequence configuration the ablations found, and the error structure dominating every residual was predicted from the class means before any network was trained. The CNN is carried forward at 0.9110 ± 0.0007 macro-F1 on test.

The ablations also show how much a result can depend on a choice made before any measurement: the Transformer's apparent tie with the MLP was an artefact of its default tokenisation, and disappeared once the representation was varied. That is the clearest direction for further work, since only two of the five architectures were ablated this way and the remaining three were never compared at more than one view of their input. The other direction is a loss reweighted towards Shirt, the one class that bounds macro-F1 for all five models.

4 AI Usage Disclosure

Claude and Gemini were used to decide what the exploratory analysis should cover, to design and scaffold the MLP, to review the training code, to write the model tests, and to draft the experiment plan. No prompt logs were retained, so each entry carries a prompt summary in their place, and no result or sentence in this report was accepted from a tool without the verification recorded below.

Tool	Claude Opus 5
Used by	Le Pham Tien Long
Task	Deciding which aspects of the dataset the EDA should cover
Prompt summary	Asked which aspects of Fashion-MNIST the exploratory analysis should cover and whether the current coverage was sufficient.
AI contribution	Proposed adding class overlap, pixel correlation against distance, and the row, column, and patch sequence profiles, and pointed out that input size and imbalance are required.
Student verification	Executed the notebook end to end, checked every figure against its printed output, and corrected two conclusions the measurements did not support.
Affected files/sections	`notebooks/eda.ipynb`, Dataset and EDA
Responsible member	Le Pham Tien Long

Tool	Claude Opus 5
Used by	Le Pham Tien Long
Task	Reviewing the training code for correctness
Prompt summary	Asked whether the training code was correct and whether it met the acceptance criteria of the issue.
AI contribution	Found that the run directory was written before the normalization statistics were computed, that metrics were rounded before checkpoint selection, and that model factories silently accepted unknown arguments; drafted the corrections.
Student verification	Ran the pipeline from two different working directories, confirmed the written run configuration holds the statistics, and retrained both models to confirm the reported metrics are unchanged.
Affected files/sections	`src/train.py`, `src/data.py`, `src/models/`
Responsible member	Le Pham Tien Long

Tool	Claude Opus 5
Used by	Le Pham Tien Long
Task	Writing pytest cases for the model contract
Prompt summary	Asked for pytest cases covering output shape, parameter count, and rejection of invalid model arguments.
AI contribution	Drafted the model test suite, 18 cases across both models.
Student verification	Ran the suite, then checked a claim the assistant made about flattening being unconditional; it was wrong for unbatched input, so the guard was kept and a regression test added.
Affected files/sections	`tests/test_models.py`, `src/models/`
Responsible member	Le Pham Tien Long

Tool	Gemini 3.8 Flash
Used by	Ho Minh Nhat
Task	Proposing an MLP architecture suited to the dataset
Prompt summary	Asked which hidden layer sizes, activation, and regularization suit 28x28 grayscale images with ten balanced classes.
AI contribution	Suggested two hidden layers of 512 and 256 units with ReLU and dropout, and explained the trade-off against a single wider layer.
Student verification	Trained the proposed architecture and compared its parameter count and validation macro-F1 against the linear baseline before adopting it.
Affected files/sections	`configs/mlp.yaml`, `src/models/mlp.py`, Methodology
Responsible member	Ho Minh Nhat

Tool	Gemini 3.8 Flash
Used by	Ho Minh Nhat
Task	Generating a code skeleton for the model module
Prompt summary	Asked for a skeleton of the model module that fits the registry interface, with depth, width, activation, and dropout taken from configuration.
AI contribution	Drafted the classifier class and its factory function.
Student verification	Completed and adjusted the skeleton by hand, then ran the model tests to confirm the output shape and the parameter count.
Affected files/sections	`src/models/mlp.py`
Responsible member	Ho Minh Nhat

Tool	Gemini 3.8 Flash
Used by	Ho Minh Nhat
Task	Reviewing whether the source code matches the handbook and is internally consistent
Prompt summary	Asked whether the modules satisfy the mandatory model requirements and whether they agree with each other on the batch and logits contract.
AI contribution	Compared the modules against the handbook requirements and pointed out where the interfaces disagreed.
Student verification	Re-read the relevant handbook sections and checked each point against the code, then ran the test suite.
Affected files/sections	`src/`
Responsible member	Ho Minh Nhat

Tool	Claude Opus 5
Used by	Le Pham Tien Long
Task	Drafting the experiment plan and fairness protocol
Prompt summary	Asked for an experiment plan fixing the shared protocol, the seed count, the tuning budget, and a quantitative rule for deciding when one model is better.
AI contribution	Drafted the plan, proposed the non-overlap rule on mean plus or minus one standard deviation across seeds, and pointed out that the handbook requires this content in the report rather than as a repository file.
Student verification	Checked every protocol value against the base configuration and the committed split, confirmed the per-epoch timings against the recorded runs, and confirmed each experiment maps to an existing backlog issue.
Affected files/sections	`EXPERIMENTS.md`, Experimental setup
Responsible member	Le Pham Tien Long

Code Availability

Every deliverable for this assignment is linked from the assignment page, `https://longlephamtien.github.io/DL261-Assignment/assignments/assignment-1/`, and reachable directly below.

Source code	`https://github.com/longlephamtien/DL261-Assignment/tree/main/assignments/assignment-1`
Checkpoints	`https://huggingface.co/longlephamtien/DL261-Assignment/tree/main/assignment-1/runs`
Slides	`https://github.com/longlephamtien/DL261-Assignment/blob/main/assignments/assignment-1/presentation/presentation.pdf`
Presentation video	To be added.

References

[1] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-mnist: A novel image dataset for benchmarking machine learning algorithms," *arXiv preprint arXiv:1708.07747*, 2017.

[2] K. Cho et al., "Learning phrase representations using rnn encoder–decoder for statistical machine translation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, 2014, pp. 1724–1734.

[3] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[4] A. Paszke et al., "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, 2019.

[5] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2019.